Plano de Projeto: P2P com Balanceamento de Carga Dinâmico

Visão Geral do Projeto

O objetivo deste projeto é desenvolver um sistema distribuído autônomo que demonstre o conceito de balanceamento de carga horizontal através de uma arquitetura P2P. Cada nó "Master" gerencia seu próprio conjunto de nós "Worker" (uma *Farm*). Os Masters monitoram sua carga de trabalho e, ao atingir um limiar de saturação, negociam dinamicamente o empréstimo de Workers de um Master vizinho para lidar com o excesso de requisições. A comunicação e a negociação entre os Masters devem seguir um protocolo de consenso, garantindo que a transferência de recursos seja coordenada e acordada.

O principal desafio é garantir a autonomia entre os sistemas desenvolvidos por diferentes equipes, permitindo que eles se interconectam e colaborem sem conhecimento prévio da implementação interna um do outro, apenas do protocolo de comunicação.

Objetivos

- **O1. Implementar a Arquitetura P2P:** Criar um nó Master capaz de gerenciar (iniciar, parar, monitorar) um conjunto de Workers (Farm).
- **O2. Simular Carga de Trabalho:** Desenvolver um mecanismo para simular requisições chegando a um Master, permitindo o monitoramento da carga.
- **O3. Implementar o Monitoramento de Saturação:** O Master deve ser capaz de identificar quando o número de requisições excede um limiar (threshold) pré-definido.
- **O4. Desenvolver o Protocolo de Conversa Consensual:** Projetar e implementar um protocolo robusto para que um Master saturado possa solicitar ajuda, um Master vizinho possa responder, e ambos possam coordenar o redirecionamento de Workers.
- **O5. Implementar o Redirecionamento Dinâmico de Workers:** Um Master vizinho deve ser capaz de instruir um ou mais de seus Workers a se reportarem temporariamente ao Master saturado.
- **O6. Garantir a Autonomia e Interoperabilidade:** O sistema deve funcionar em conjunto com o sistema de outra equipe, comunicando-se exclusivamente através do protocolo definido.

3. Arquitetura Proposta

O sistema será composto por três entidades principais:

1. **Nó Master:**
   - Recebe requisições de clientes (simuladas).
   - Mantém uma lista de Workers em sua Farm.
   - Distribui requisições entre seus Workers.
   - Monitora constantemente o número de requisições pendentes.
   - Inicia o "Protocolo de Conversa Consensual" quando requisições > threshold.
   - Gerencia Workers "emprestados" de outros Masters.

2. **Nó Worker:**
   - Recebe uma tarefa do seu Master atual e a processa (ex: um cálculo simples, um sleep para simular trabalho).
   - Deve ser capaz de se desconectar do seu Master original e se conectar a um novo Master quando instruído.
3. **Protocolo de Comunicação (Mensageria):**
   - A espinha dorsal do projeto. Define as regras para a negociação.
   - Será baseado em trocas de mensagens, por exemplo, via API REST, gRPC ou Sockets customizados com formato JSON.

Considerações de Implementação

- **Message Delimiter:** Em um stream de TCP, você precisa de uma forma de saber onde uma mensagem JSON termina e a próxima começa. A abordagem mais simples é enviar um caractere de nova linha (\n) após cada objeto JSON. O receptor lê o socket até encontrar um \n, e então processa o que recebeu como um JSON completo.
- **Escuta Contínua:** Cada Master e Worker deve rodar em um loop infinito, escutando por novas conexões (se for servidor) ou novas mensagens em conexões existentes.
- **Threads/AsyncIO:** É fundamental usar concorrência (threads ou programação assíncrona) para que um Master possa se comunicar com múltiplos outros Masters e Workers simultaneamente sem bloquear seu processo principal.
- **Protocolos de Comunicação devem seguir o padrão estabelecido em sala.**

PAYLOAD OFICIAL:

|1| HEARTBEAT - Worker A1 - Master A - Verificação de Atividade.

|1.1| Worker A1 → Master A - Worker A1 pergunta ao Servidor B se ele está ativo.

PAYLOAD:

```
{
    "SERVER_UUID": "Master_A",
    "TASK": "HEARTBEAT"
}
```

|1.2| Master A → Worker A1 - Servidor B responde que está ativo.

PAYLOAD:

```
{
    "SERVER_UUID": "Master_A",
    "TASK": "HEARTBEAT",
    "RESPONSE": "ALIVE"
}
```

Sprint: Implementação do Mecanismo de Heartbeat (Prazo Entrega: Ver Atividade no Sala Online)

Estabelecer a comunicação base entre o Worker e o Master, garantindo que o Worker consiga verificar se o seu "mestre" está ativo através de trocas de mensagens JSON via TCP.

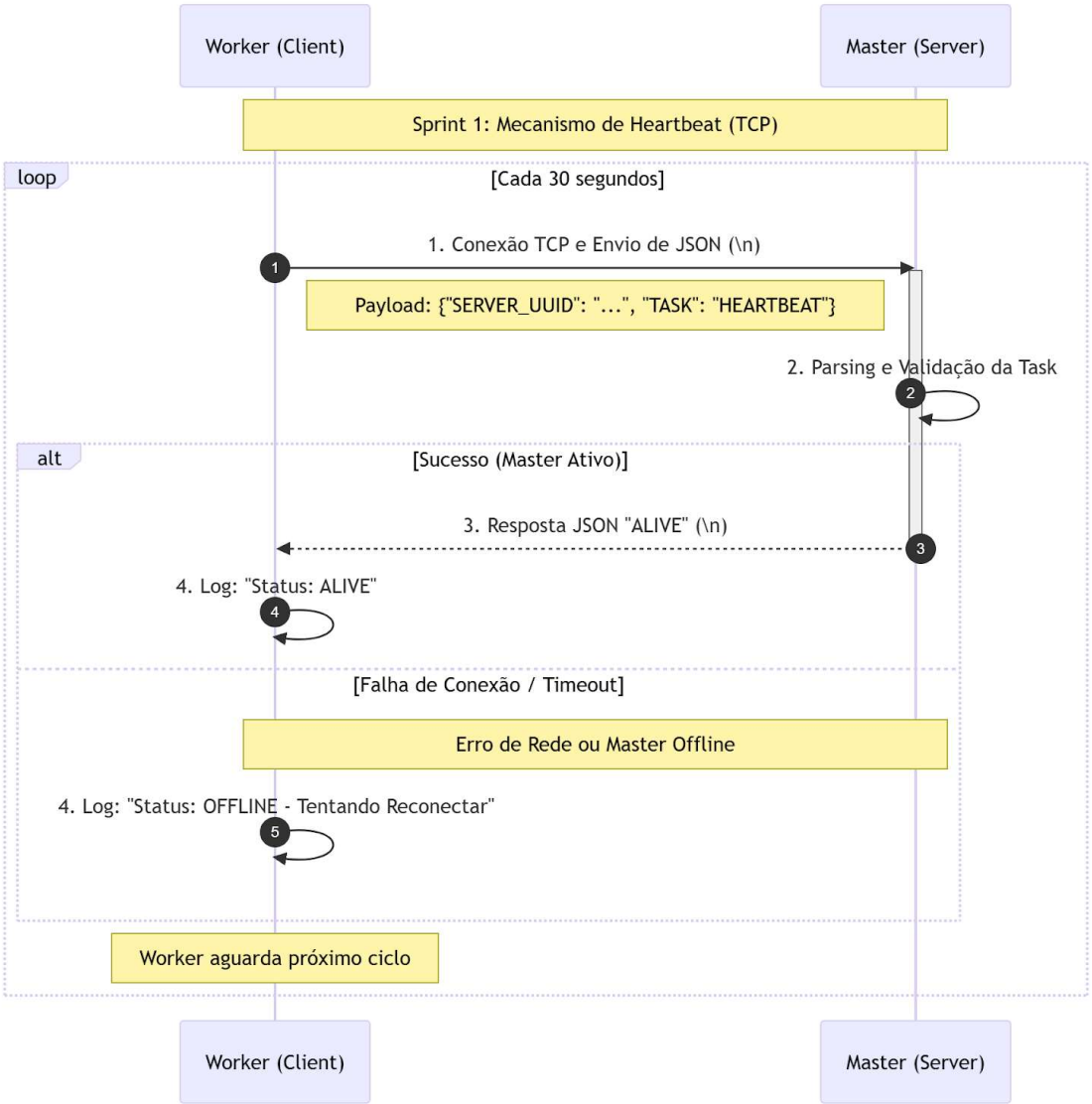
1. Backlog de Tarefas (To-Do)

- **Tarefa 01: Infraestrutura de Comunicação TCP**
  - Configurar o Master para atuar como servidor, escutando em uma porta definida.
  - Configurar o Worker para atuar como cliente, iniciando a conexão.
  - Padrão de Mensagem: Implementar o delimitador de nova linha (\n) ao final de cada JSON para garantir que o receptor identifique o fim da mensagem no stream TCP.
- **Tarefa 02: Lógica de Requisição (Worker → Master)**
  - Desenvolver a função no Worker para disparar o payload de verificação.
  - Payload de Envio: {"SERVER_UUID": "...", "TASK": "HEARTBEAT"}.
- **Tarefa 03: Lógica de Resposta (Master → Worker)**
  - Implementar no Master a capacidade de interpretar a tarefa "HEARTBEAT" e devolver a confirmação imediata.
  - Payload de Resposta: {"SERVER_UUID": "...", "TASK": "HEARTBEAT", "RESPONSE": "ALIVE"}.
- **Tarefa 04: Concorrência e Resiliência**
  - Utilizar Threads ou AsyncIO no Master para que o atendimento de um Heartbeat não bloqueie outras operações (como o processamento de tarefas reais).
  - Criar um loop no Worker para repetir essa verificação em intervalos regulares (ex: a cada 10 segundos).

2. Definição de "Pronto" (DoD)

A entrega será considerada concluída quando:

1. O Worker conseguir abrir uma conexão TCP com o Master.
2. O Master receber o JSON, realizar o *parsing* e identificar o comando de Heartbeat.
3. O Worker receber a confirmação "ALIVE" e conseguir imprimir isso no log.
4. A conexão for mantida ou reestabelecida corretamente sem travar os processos.



Sprint 2: Comunicação de Tarefas e Apresentação de Workers

Implementar o fluxo completo de ciclo de vida de uma tarefa, desde a apresentação do Worker (identificando sua origem) até o processamento e a confirmação final de recebimento do status pelo Master.

1. Backlog de Tarefas (To-Do)

- **Tarefa 01: Lógica de Apresentação e Identificação (Worker → Master)**
  - Implementar no Worker a capacidade de se apresentar ao Master enviando seu UUID.
  - *Diferencial de Origem: O Worker deve ser capaz de enviar o payload de "Emprestado" caso esteja vinculado a um Master original diferente.*
  - Payloads (2.1 e 2.1b): `{"WORKER": "ALIVE", "WORKER_UUID": "..."}` ou `{"WORKER": "ALIVE", "WORKER_UUID": "...", "SERVER_UUID": "..."}`.
- **Tarefa 02: Distribuição de Carga e Gestão de Fila (Master → Worker)**
  - Configurar o Master para gerenciar uma fila (queue) de tarefas pendentes.
  - Ao receber um pedido de tarefa, o Master deve verificar a fila:
    - Com Tarefa (Payload 2.2): Enviar `{"TASK": "QUERY", "USER": "..."}`.
    - Fila Vazia (Payload 2.3): Enviar `{"TASK": "NO_TASK"}`.
- **Tarefa 03: Simulação de Processamento e Relatório de Status (Worker → Master)**
  - Desenvolver no Worker o "executor": ao receber uma `QUERY`, ele deve simular um processamento (ex: `sleep` aleatório ou cálculo).
  - Após o trabalho, o Worker deve reportar o resultado.
  - Payload (2.4): `{"STATUS": "OK | NOK", "TASK": "QUERY", "WORKER_UUID": "..."}`.
- **Tarefa 04: Mecanismo de Confirmação (ACK) e Persistência (Master → Worker)**
  - Implementar no Master o recebimento do status e a resposta de confirmação imediata para liberar o Worker.
  - Payload (2.5): `{"STATUS": "ACK"}`.
  - Garantir que o Master registre (log) qual Worker (local ou emprestado) concluiu qual tarefa.

2. Definição de "Pronto" (DoD)

A entrega será considerada concluída quando:

1. O Worker realizar o "handshake" de apresentação com sucesso (enviando UUID).
2. O Master distribuir uma tarefa real da fila ou informar corretamente que não há tarefas.
3. O Worker processar a tarefa e o Master receber o status `OK` ou `NOK`.
4. O Worker receber o `ACK` final do Master, fechando o ciclo de comunicação sem erros de parsing ou perda de mensagens no stream TCP.
5. O sistema tratar corretamente a presença (ou ausência) do campo `SERVER_UUID` no

payload de apresentação.

Definição do Payload Padrão

Para que o sistema de uma equipa "A" fale com o sistema da equipa "B", todos os pacotes devem seguir rigorosamente esta estrutura JSON, terminando sempre com o caractere `\n`.

1. Solicitação de Tarefa (Worker → Master)

- Campos obrigatórios: `WORKER`, `WORKER_UUID`.
- Campo opcional: `SERVER_UUID` (usado apenas se o worker for "emprestado").

```
{
    "WORKER": "ALIVE",
    "WORKER_UUID": "string",
    "SERVER_UUID": "string"
}
```

2. Entrega de Tarefa (Master → Worker)

Caso haja tarefa:

```
{
    "TASK": "QUERY",
    "USER": "string"
}
```

Caso não haja tarefa:

```
{
    "TASK": "NO_TASK"
}
```

3. Reporte de Status (Worker → Master)

Campos obrigatórios: `STATUS` (valores: "OK" ou "NOK"), `TASK`, `WORKER_UUID`.

```
{
    "STATUS": "OK",
    "TASK": "QUERY",
    "WORKER_UUID": "string"
}
```

4. Confirmação Final (Master → Worker)

Campo único: STATUS (valor fixo: "ACK").

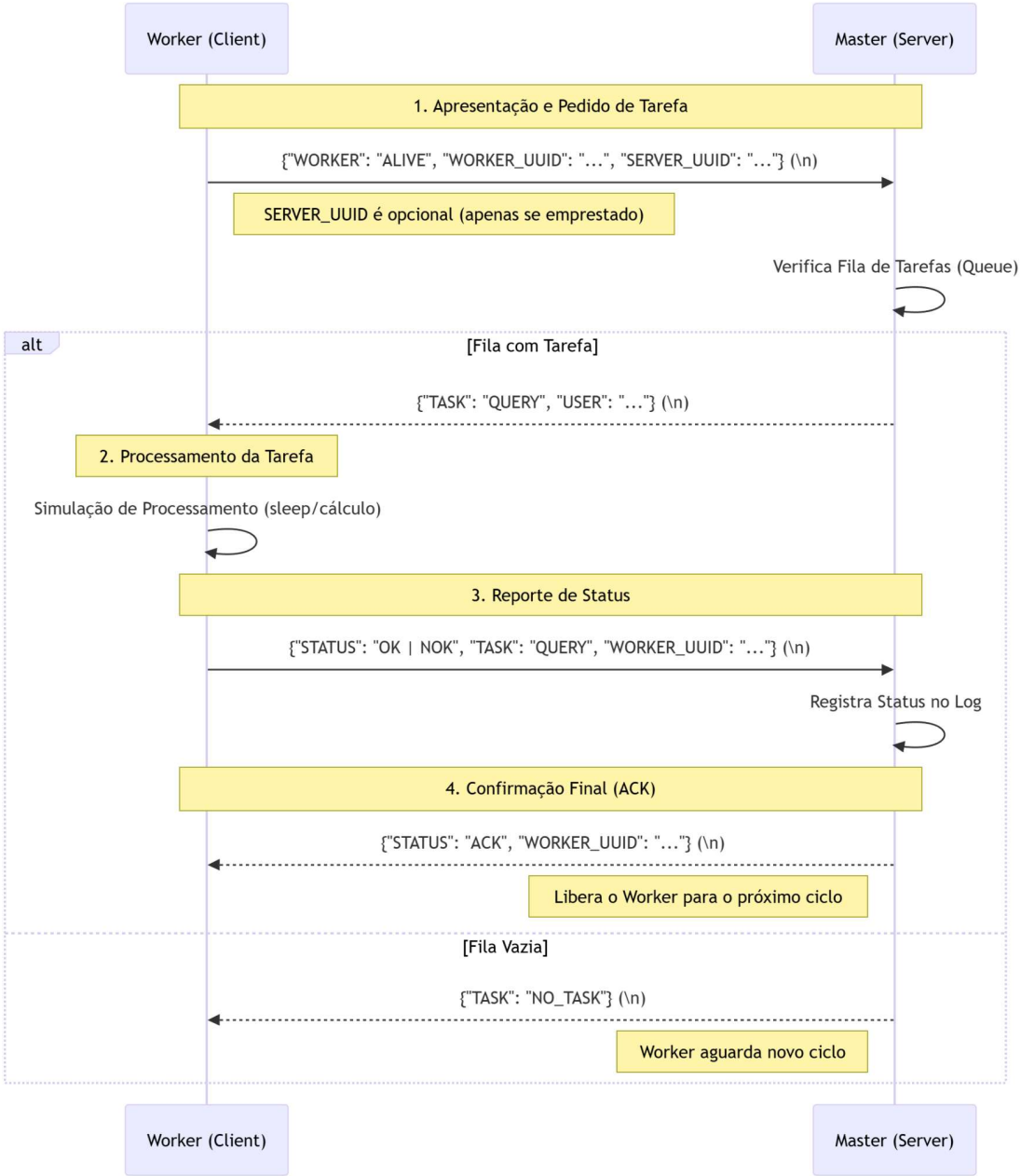
```
{
    "STATUS": "ACK",
    "WORKER_UUID": "string"
}
```

Notas de Implementação:

1. ***Strict Parsing:*** Os Masters devem ser programados para ignorar campos desconhecidos no JSON (para permitir extensões futuras), mas devem falhar se os campos obrigatórios acima estiverem ausentes.
2. ***Case Sensitivity:*** Todos os valores de controle (ALIVE, QUERY, NO_TASK, OK, NOK, ACK) devem ser tratados em CAIXA ALTA.
3. ***Timeout:*** O Worker deve aguardar a resposta do Master por no máximo 5 segundos antes de considerar a conexão perdida e tentar uma nova conexão.

ID	Cenário de Teste	Ação do Worker (JSON)	Resposta Esperada do Master	Critério de Sucesso
CT01	Apresentação Worker Local	{"WORKER": "ALIVE", "WORKER_UUID": "W-123"}	{"TASK": "QUERY", "USER": "Michel"}	Master identifica Worker local e entrega uma tarefa da fila.
CT02	Apresentação Worker Emprestado	{"WORKER": "ALIVE", "WORKER_UUID": "W-999", "SERVER_UUID": "Master-B"}	{"TASK": "QUERY", "USER": "Julia"}	Master reconhece que o Worker pertence a outro nó, mas atribui tarefa normalmente.

CT03	Fila de Tarefas Vazia	{"WORKER": "ALIVE", "WORKER_UUID": "W-123"}	{"TASK": "NO_TASK"}	O Master responde corretamente quando não há trabalho pendente.
CT04	Reporte de Sucesso	{"STATUS": "OK", "TASK": "QUERY", "WORKER_UUID": "W-123"}	{"STATUS": "ACK"}	O Master processa o sucesso e liberta o Worker com um ACK.
CT05	Reporte de Falha	{"STATUS": "NOK", "TASK": "QUERY", "WORKER_UUID": "W-123"}	{"STATUS": "ACK"}	O Master regista a falha, mas ainda assim envia o ACK para confirmar o recebimento do status.


Worker (Client)
Master (Server)
1. Apresentação e Pedido de Tarefa
{"WORKER": "ALIVE", "WORKER_UUID": "...", "SERVER_UUID": "..."} (\n)
SERVER_UUID é opcional (apenas se emprestado)
Verifica Fila de Tarefas (Queue)
alt
[Fila com Tarefa]
{"TASK": "QUERY", "USER": "..."} (\n)
2. Processamento da Tarefa
Simulação de Processamento (sleep/cálculo)
3. Reporte de Status
{"STATUS": "OK | NOK", "TASK": "QUERY", "WORKER_UUID": "..."} (\n)
Registra Status no Log
4. Confirmação Final (ACK)
{"STATUS": "ACK", "WORKER_UUID": "..."} (\n)
Libera o Worker para o próximo ciclo
[Fila Vazia]
{"TASK": "NO_TASK"} (\n)
Worker aguarda novo ciclo
Worker (Client)
Master (Server)